

Instituto Tecnológico de Costa Rica
Escuela de ingeniería en computación
IC-4700 Lenguajes de Programación
P03 - Operación Atlas

Estudiante:

Sidney Salazar Jiménez

2023282075

Profesor(a) a cargo:

Allan Rodríguez

I Semestre 2026

Manual de usuario

Instrucciones de compilación:

Para compilar el programa es necesario tener instalado con anterioridad SWI-prolog y un ambiente en donde se ejecutarán las consultas como lo podría ser el mismo SWI-prolog o Visual Studio Code.

ejecución y uso:

Para ejecutar el sistema, descargar el archivo desde el repositorio del proyecto, después se debe acceder a la carpeta Backend/prolog desde el ambiente elegido. Se debe escribir en la terminal lo siguiente:

```
swipl juego.pl  
1 ?- consult('juego.pl').
```

Todas las consultas se manejan al invocarlas directamente desde la terminal.

Pruebas de funcionalidad

mover(Modulo).

Desplaza al jugador desde su posición actual hacia otro modulo.

```
2 ?- mover(enfermeria).  
Te moviste hacia enfermeria
```

tomar(Artefacto).

Agrega el artefacto a la lista de artefactos logrados, siempre que está en el modulo actualmente. Al moverse a un modulo, no necesariamente se toma el artefacto, se debe hacer explícitamente.

```
3 ?- tomar(traje_espacial).  
Tomas el objeto traje_espacial y sigues tu camino.  
true .
```

usar(Artefacto).

Permite registrar el uso de un artefacto, para la cual desbloqueará el acceso a Modulos que lo requieran.

```
14 ?- usar(traje_espacial).  
Haz usado el objeto traje_espacial y desbloqueaste modulo_energia  
true .
```

reparar(Sistema).

Permite restaurar sistemas críticos de la estación espacial.

```
27 ?- reparar(energia).  
El sistema energia ha sido restaurado.  
true .
```

rescatar(Tripulante).

Permite rescatar un miembro de la tripulación.

```
37 ?- rescatar(kai).  
Acabas de salvar al tripulante kai  
true .
```

puedo_ir(Hacia).

Determina si el jugador puede moverse desde su ubicación actual hacia un destino.

```
39 ?- puedo_ir(puente_mando).  
Puedes avanzar hacia puente_mando  
true .
```

donde_esta(Artefacto).

Indica lugar del artefacto.

```
40 ?- donde_esta(fusible).  
Encuentras el artefacto fusible en tu inventario.  
true .
```

que_tengo.

Indica la lista de artefactos logrados por el jugador.

```
41 ?- que_tengo.  
Encuentras en tu inventario los artefactos:  
- tarjeta_seguridad  
- fusible  
- traje_espacial  
true.
```

modulos_visitados.

Indica los modulos por los que se ha transitado.

```
42 ?- modulos_visitados.  
Lugares visitados:  
- modulo_energia  
- laboratorio  
- enfermeria  
- puente_mando  
true.
```

ruta(Inicio, Fin, Camino).

Encuentra una ruta lógica entre dos modulos utilizando backtracking. El resultado es una lista que representa el camino entre los dos puntos.

```
43 ?- ruta(enfermeria, laboratorio, R).  
R = [enfermeria, puente_mando, laboratorio] []
```

como_gano.

Encuentra las posibles formas de completar el juego considerando:

- rutas disponibles,
- artefactos requeridos,
- reparaciones necesarias,
- rescates pendientes,
- y restricciones lógicas.

```

44 ?- como_gano.
+. ¿Como Gano? .+

Sistemas a restaurar:
[LISTO] energia en modulo_energia (requiere: [fusible])
[PENDIENTE] comunicaciones en laboratorio (requiere: [fusible, traje_espacial])

Tripulantes a rescatar:
[RESCATADO] elena en modulo_energia (necesita sistemas: [energia])

Rutas desde posicion actual:
Hacia elena: [enfermeria, puente_mando, laboratorio, modulo_energia]
true []

```

verifica_gane.

Verifica si el jugador ha cumplido todas las condiciones de victoria.

En caso de gane, debe mostrar:

1. Ruta realizada.
2. Lista final de artefactos logrados.
3. Sistemas reparados.
4. Tripulación rescatada.
5. Condición de victoria alcanzada.

```

*+. OPERACION ATLAS COMPLETADA! .+*

Ruta realizada:
- modulo_energia
- laboratorio
- enfermeria
- puente_mando

Artefactos logrados:
- tarjeta_seguridad
- fusible
- traje_espacial

Sistemas reparados:
- comunicaciones
- energia

Tripulacion rescatada:
- kai
- elena

Condicion de victoria: TODOS LOS OBJETIVOS CUMPLIDOS
51 ?- verifica_gane.[]

```

Descripción del problema.

El proyecto plantea un juego en donde se deben seguir ciertas condiciones lógicas para ganar, para cumplir estas condiciones, el jugador debe moverse entre varios módulos para recolectar artefactos, para ya sea reparar los sistemas o rescatar a los tripulantes. Esto implica el asegurar una experiencia lo más congruente posible a hechos lógicos que la mayoría de juegos de este tipo tienen,

como lo pueden ser asegurar las conexiones en ambos sentidos entre los módulos, asegurando que el jugador no pueda saltar de un módulo a otro al que no exista conexión directa. También se desea que el sistema ayude al usuario a guiarse en el entorno en el que está, por lo que funciones que hagan una lectura exhaustiva de la información solicitada del juego son indispensables para mantener la experiencia lo suficientemente informativa para el avance del jugador.

Diseño del programa: decisiones de diseño, algoritmos usados.

Se decidió mantener separado el proyecto en tres archivos: “[hechos.pl](#)”, “[juego.pl](#)” y “[Auxiliares.pl](#)”. Esto con el fin de hacer una diferenciación entre los elementos principales del programa, y en general, para mantener la modularización del sistema, la cual busca que el código fuente sea una lectura sin complicaciones. Se creó el archivo “[Auxiliares.pl](#)” con el objetivo de abstraer las funciones más utilizadas a lo largo del proyecto, pero que no necesariamente fueron solicitadas en la documentación del mismo. El archivo “[hecho.pl](#)” es usado como una plantilla para permitir la modificación de los hechos del juego sin afectar las reglas del juego. En “[juego.pl](#)” es donde se concentra la mayoría del desarrollo del proyecto, en este están todas los predicados y reglas que lo componen.

En cuanto a las decisiones respecto a cómo se debía manejar los elementos que componen al juego, se pueden destacar el uso de inferencias para asegurar la conectividad entre módulos, esta manera resulta efectiva para evitar el uso de predicados repetidos (que no serían efectivos si se cambian los hechos del juego). El uso de estados mutables se decidió por motivo del cambio constante que implica el avance del jugador en el juego a los hechos estáticos, esto se resalta más en el predicado de sistema, que al hacerlo dinámico permite cambiar el estado del mismo, sin necesidad de mantener en una tabla de estados, y permitiendo la consulta directa desde el predicado. También se decidió manejar las consultas de precondiciones en un sintaxis tipo cascada, con el fin de mantener las reglas lo más claras posible. En cuanto a cómo se encuentra la ruta de un punto a otro, se decidió usar un algoritmo de búsqueda por profundidad (DFS) que aprovecha el backtracking natural de Prolog. En dos predicados en particular, se vio la necesidad de saber si todos los elementos de una lista estaban en otra, por lo que se creó el predicado de coincidencias que satisface esta necesidad.

7. Análisis de predicados:

Mover: Permite el traslado de un módulo a otro.

Para este predicado se tuvieron que realizar varias validaciones al mismo tiempo, como lo podría ser la verificación de lugares bloqueados, revisión de requisitos alcanzados por el usuario, actualización del historial de movimientos, verificación de ramas, todo mientras se mantenía informado al usuario de avance hacia los siguientes módulos, y en caso que no se haya avanzado de un módulo a otro, el explicar al usuario sobre el modulo bloqueado.

Reparar: Restaura sistemas declarados con fallos.

De nuevo se validaron múltiples requisitos, como lo es la ubicación de usuario respecto a la del sistema dañado, validación de estados, comprobación de reparaciones previas, y validación de subconjuntos mediante Coincidencias.

Como gano: Generación de una guía que enlista todas la acciones necesarias para ganar el juego.

La más complicada en cuanto a semántica, donde se evaluaron la mayoría de conocimientos previos del sistema, en este se comprobaron que objetivos de reparación y rescate, objetos necesarios para lograr esto y rutas disponibles hacia los objetivos. Se hizo un sobreuso de forall que tuvo por objetivo recorrer todos lo objetivos definidos. Además utilizando el algoritmo de búsqueda de rutas, lo cual le agrego mas carga al depender de un algoritmo tan importante.